



Cargo [EN]

▼ `cargo new <project_name>`

Creates a new directory with the standard Rust project architecture.

▼ `cargo init`

Converts the current directory into a Rust project.

▼ `cargo check`

Checks that your code compiles and is memory-safe, without generating an executable.

▼ `cargo build`

Compiles the code and generates an unoptimized executable in the `target/debug/` directory.

▼ `cargo run`

Compiles the code and immediately runs the executable.

▼ `cargo build --release`

Compiles the code with full speed optimizations (essential for a game engine like StellarHash). The executable is generated in `target/release/`. Compilation time is significantly longer.

▼ `cargo add <crate_name>`

Adds a dependency to the `Cargo.toml` file.

▼ `cargo remove <crate_name>`

Cleanly removes a dependency.

▼ `cargo update`

Updates all project dependencies to their latest compatible minor versions and refreshes the `Cargo.lock` file.

▼ `cargo test`

Runs all unit and integration tests for your project.

▼ `cargo fmt`

Formats all of your source code according to official Rust standards.

▼ `cargo clippy`

Runs an advanced static analysis to suggest syntax and performance improvements, and catch logical errors.

▼ `cargo clean`

Removes the `target/` directory generated by builds. Very useful if you need to free up disk space (Rust build directories can quickly weigh several gigabytes).

▼ `cargo doc --open`

Generates beautiful HTML documentation from your own code's comments (as well as those of all your dependencies) and opens it directly in your web browser.

▼ `cargo bench`

The command compiles and runs your Rust project's performance tests to precisely measure the speed and efficiency of your code.

▼ `cargo bench --bench <file name of bench>`

This command allows you to execute a specific benchmark file to measure its performance without having to compile and run all the other tests in the project.